CSC2103

DATA STRUCTURES AND ALGORITHMS

Group Assignment

Visualization of [Binary Search Tree, Self-Balancing Tree (AVL Tree) and B-Tree (2-3-4 Tree)]

By:

Liang Yrenn Da（22067128）

Ooi Jin Yon（22039069）

Isabelle Yau Li Shuang（23021165）

Peggy Liow En Ki（23020852）

Leow Yan Yi（23020324）

Department of Smart Computing and Cyber Resilience

School of Engineering and Technology

Sunway University

Table of Contents

Part 1: Introduction 3

Part 2: Problem 1 Implementation: **Binary Search Tree** 4

Part 3: Problem 2 Implementation: **Self-Balancing Tree (AVL Tree)** 23

Part 4: Problem 3 Implementation: **B-Tree (2-3-4 Tree)** 39

Part 5: Individual Member Contribution 43

Part 6: Challenges 44

Part 7: Conclusion 46

Part 8: References 47

Part 1: Introduction

We have selected three problems, which are **Binary Search Tree (BST), AVL Tree and B-Tree (2-3-4 Tree)**. The reason we chose 3 different trees is because three of them are balanced trees, where the BST can become a height-balanced tree, AVL tree is self-balanced tree and Balanced Tree (B-Trees) is a perfectly balanced tree. Thus, we would like to test by implementing the code to see how they work.

Firstly, we choose BST since it exists as a basic concept of binary search and node traversal in data structures. Although it follows a hierarchical organization, it may also have the probability of having poor performance since it might become unbalanced. Nevertheless, BST is still available to represent a height-balanced structure by implementing balancing techniques properly.

Other than that, we chose AVL Tree because it has a consistent performance and efficient search operation. It is also a variant of the BST that can be self-balanced. AVL tree has a unique function which is always to preserve a height difference allocated at maximum one between the left and right subtree of any node. This function can solve the unbalanced issue faced by BST. It provides consistent efficiency during operations.

Furthermore, we chose the B-Tree because we are interested in this tree as it is one of the most efficient and perfectly balanced trees compared to the others. It represents a multi-way search tree that enables perfect balance. Thus, it is suitable and able to handle large datasets with efficiency. It can have a maximum of three keys within one node and use minimum height to maintain sorted data. With these features, it also becomes the most common tree that is used in database and file systems.

Last but not least, the **programming language** that we chose is **Python** which we only learned before. It provides convenience to us due to its accessibility, rich library support and simplicity. Besides, it also allows us to implement these complex data structures effectively. This report aims to enhance our knowledge of data structures by addressing the practical challenges with these trends.

Part 2: Problem 1 Implementation: Binary Search Tree

Binary Search Tree (BST) is a data structure for organizing and storing data in a sorted manner. Each node in BST has at most two children, a left child and a right child, with the left child containing values less than the parent node and the right child containing values greater than the parent node.

To be specific, these are the rules:

- **Node Formation:** Each node in a tree contain value, the tree can have only one root node that is on the topmost of the entire tree and at most 2 children node, the children node refers as left child of right child.
- **Left Subtree Rule:** Every node in the left subtree of a given node must have a value that is smaller than the value of that node.
- **Right Subtree Rule:** Every node in the right subtree of a given node must have a value that is larger than the value of that node.
- **Unique Keys:** Each node in a **Binary Search Tree (BST)** must be a unique number. Moreover, duplicate keys cannot be allowed in a **BST**.
- **Recursive Structure:** The left and right subtrees of **BST** can be their own structure. It is design to simplify the operations such as find, insert and delete.

In this implementation, we have randomly selected 20 numbers, integers ranging between 1 to 200, as our dataset. Table 2.1 below shows the dataset used to implement our Binary Search Tree.

68	18	32	70	41
8	13	103	123	89
19	29	100	121	9
156	62	60	15	144

Table 2.1.1 Random Number

We have 4 functions in this program: insert, delete, find and clear tree. Users can only enter integer values ranging between 1 to 200. Otherwise, an error message will be displayed.

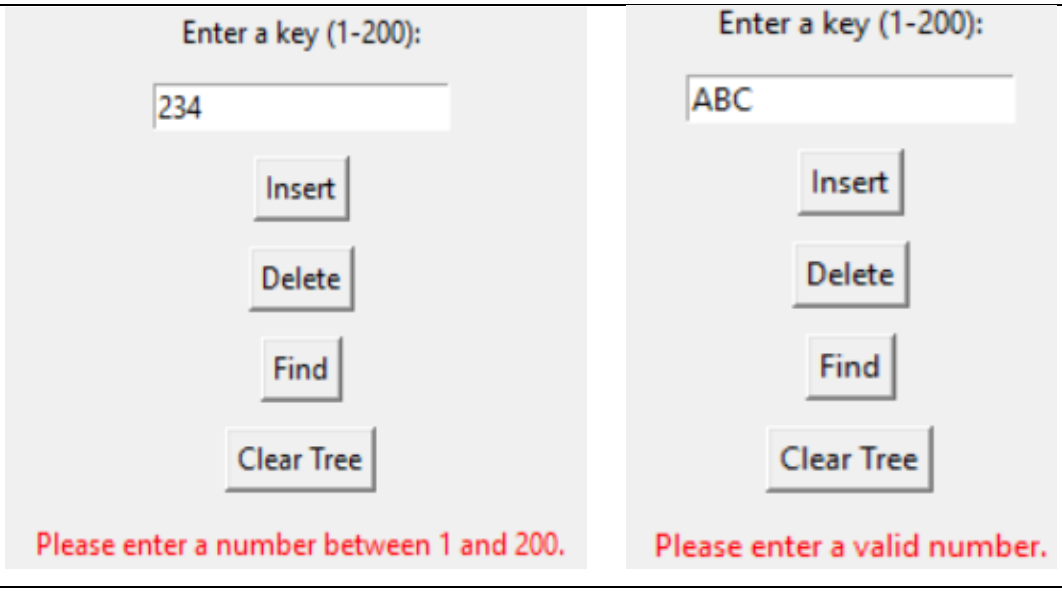


Table 2.1.2 Invalid key

Insertion

Step 1 – Insert the 1st number: **68**.

We will begin by entering the first random number, 68. When 68 is entered, it will be placed on the top of the tree, which we call as root node.

Table 2.1.3 Insert 1st key

Step 2 – Insert 2nd and 3rd numbers: **18, 32**.

When the second number (18) is entered, the algorithm compares it to the root node (68). Since 18 is smaller than 68, it is placed in the left subtree. The next number (32) is then compared to 68. Since 32 is smaller than 68 but larger than 18, it will be placed in the right subtree of 18, creating a balanced structure with 18 as the parent node and 32 as its child node.

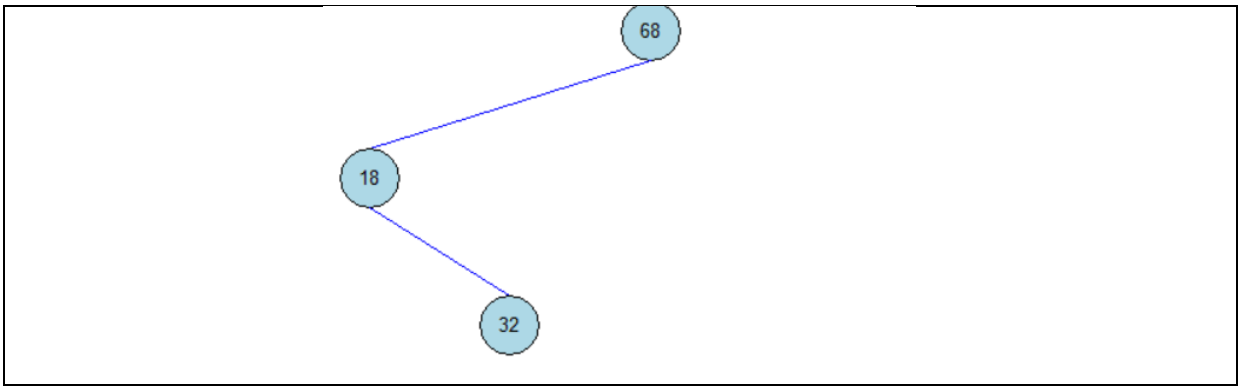


Table 2.1.4 Insert 2nd and 3rd key

Step 3 – Insert 4th number: **70**.

Continuing with our fourth number, it appeared to be larger than 68. Thus, is placed in the right subtree of 68.

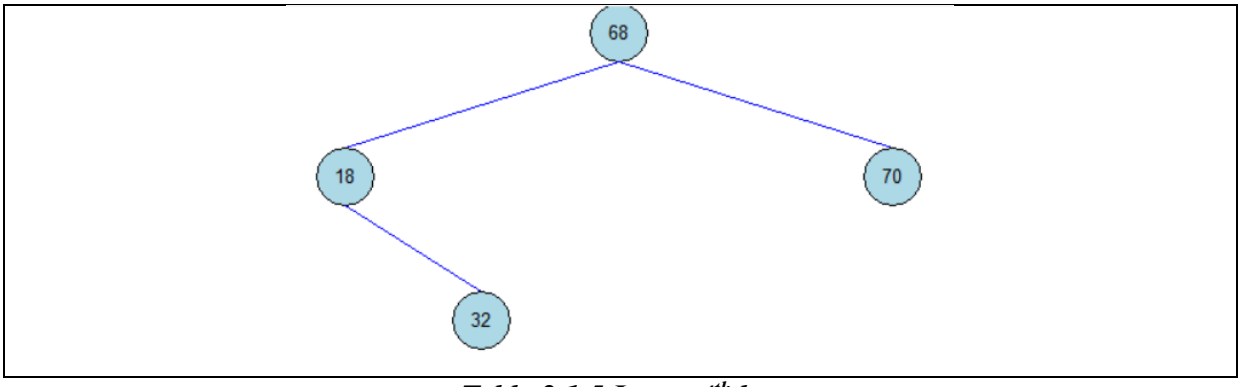


Table 2.1.5 Insert 4th key

Step 4 – Insert 5th to 10th numbers: **41, 8, 13, 103, 123, 89**.

After inserting the 10th number, the resulting tree will still follow the rules of the algorithm, maintaining the correct binary search tree structure.

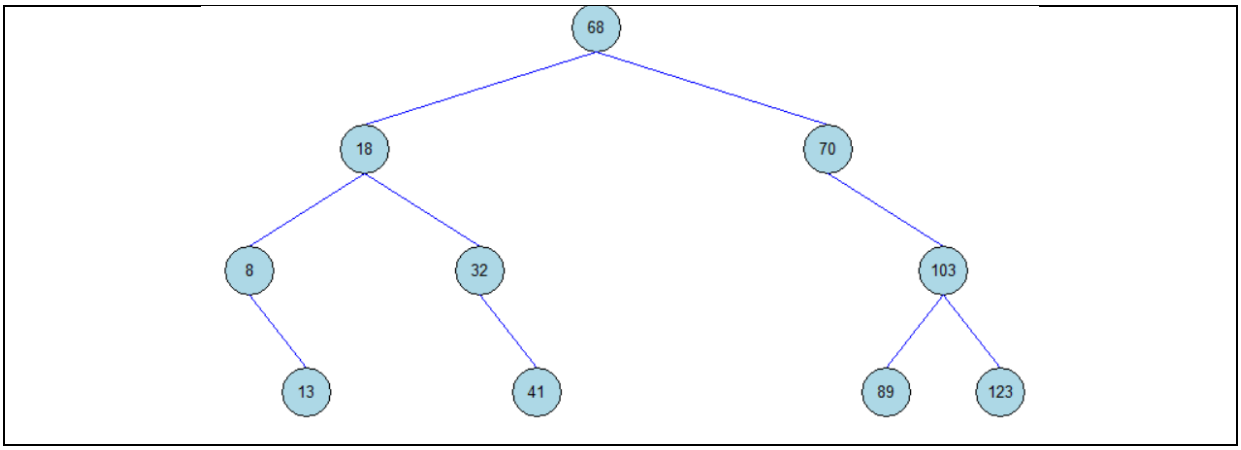


Table 2.1.6 Insert 5th to 10th key

Step 5 – Insert 11th to 20th numbers: **19, 29, 100, 121, 9, 156, 62, 60, 15, 144**

Similarly, as the 11th to 20th numbers are inserted, the tree continued to follow binary search tree algorithm with each value placed in the appropriate position.

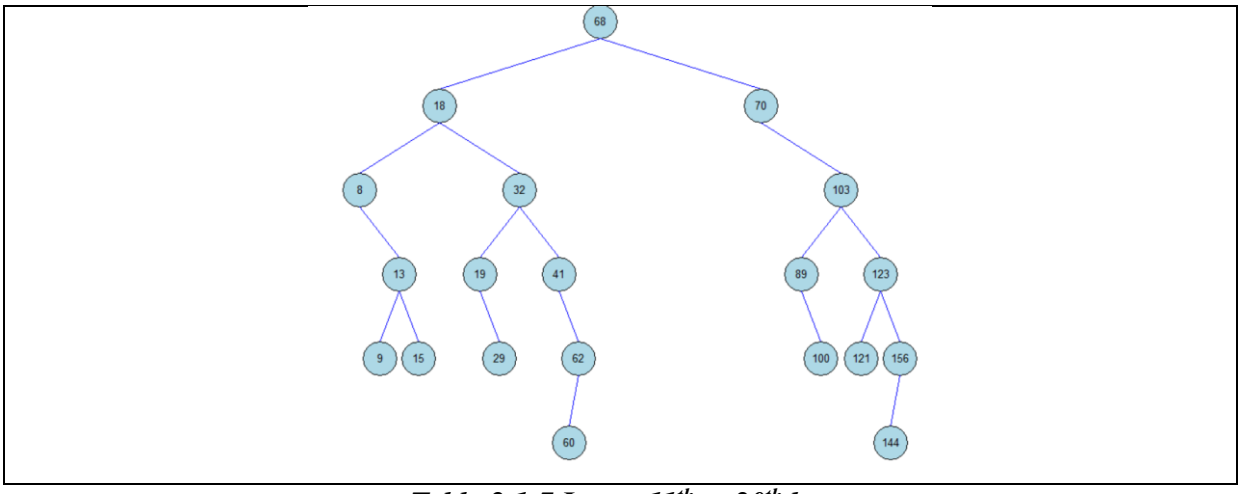


Table 2.1.7 Insert 11th to 20th key

Deletion

After going through insertion, we will test our second function which is **Deletion,** it is used to remove unwanted numbers in the tree while ensuring the properties of binary search tree. To test this, we use the tree containing the 20 nodes created during the insertion phrase.

Test 1 – Delete number 89 (within the tree).

This is the tree structure of before and after the number 89 gets deleted. After 89 is removed, number 100 moves up to take its place and is now positioned directly under number 103.

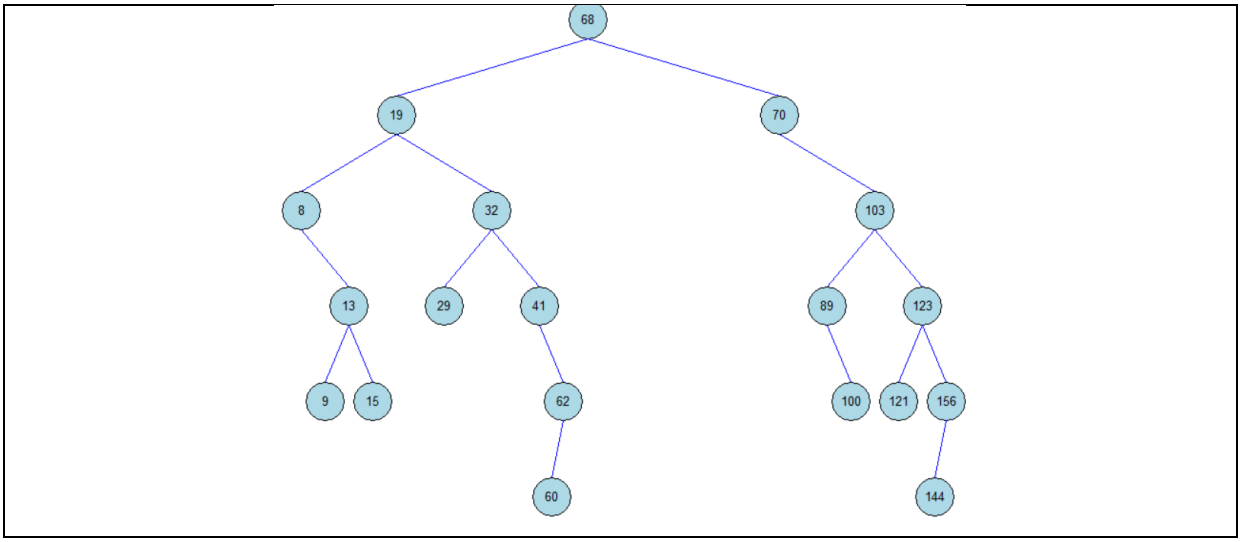


Table 2.2.1 Before Delete number 89

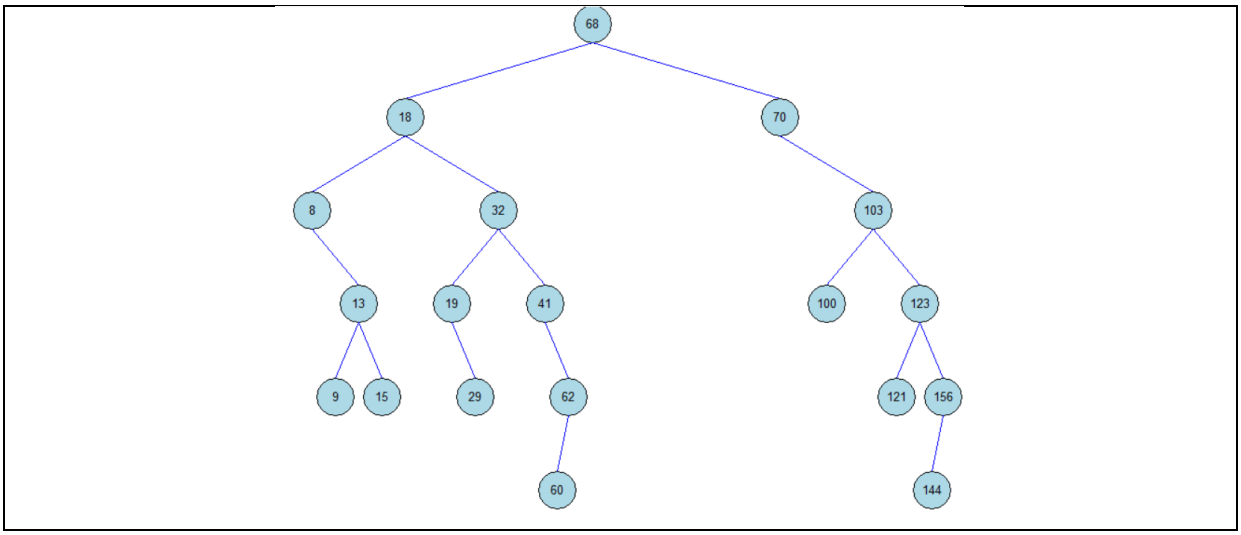


Table 2.2.2 After Delete number 89

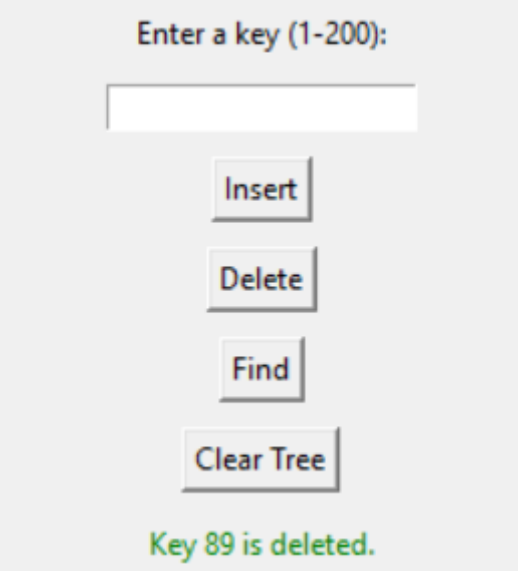


Table 2.2.3 Message label "Key 89 is deleted."

Test 2 – Delete number 123 (within the tree).

This is the tree structure of before and after the number 123 gets deleted. After 123 is removed, the three numbers below it which are 121, 156 and 144 are then rearranged based on the binary search tree properties to maintain the correct structure.

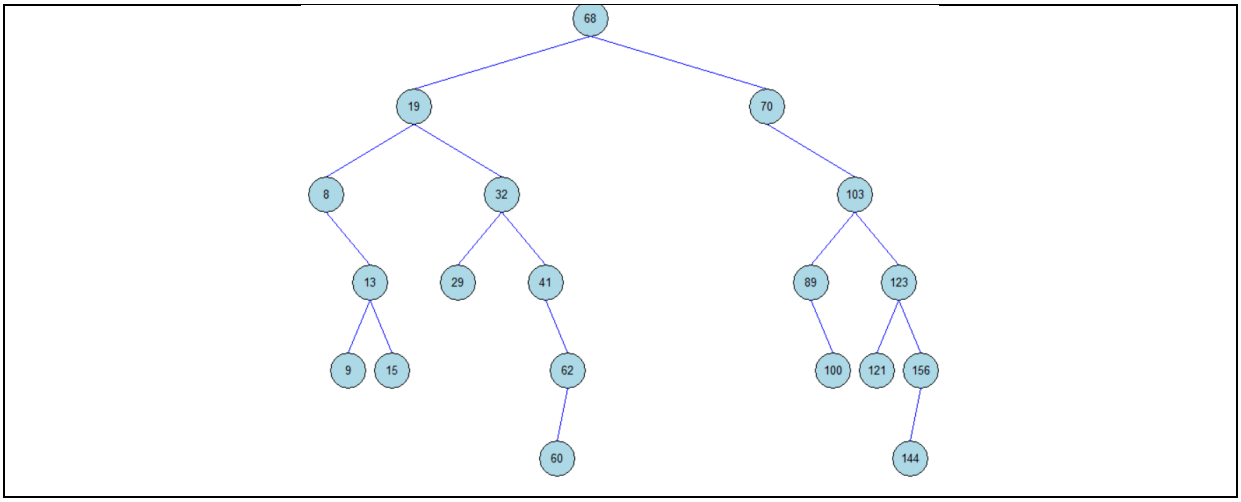


Table 2.2.4 Before Delete number 123

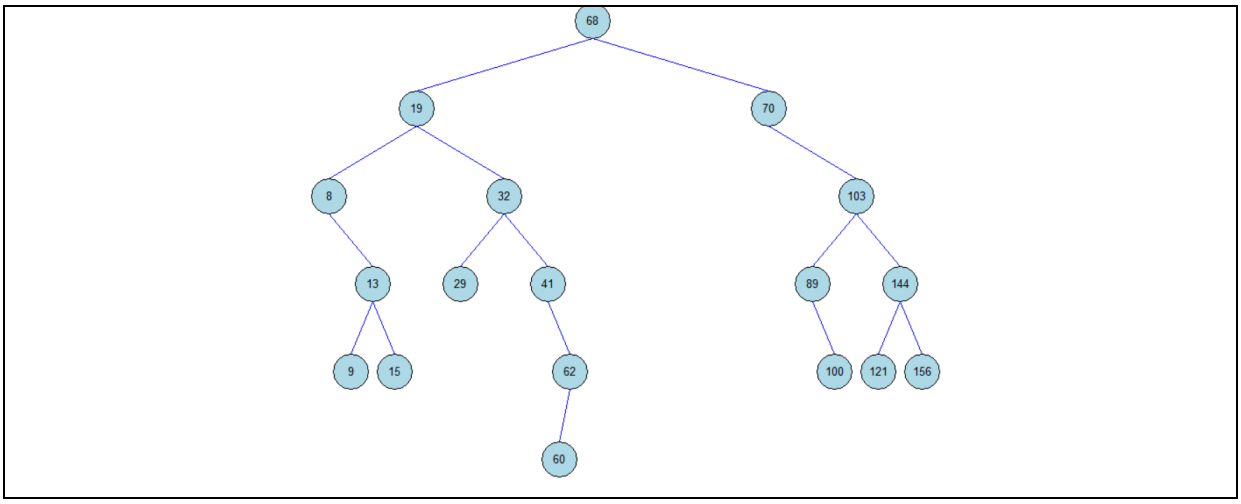


Table 2.2.5 After Delete number 123

Table 2.2.6 Message label "Key 123 is deleted."

Test 3 – Delete number 18 (not within the tree).

This is the tree structure of before and after the number 18 gets deleted. Since the number did not exist in the tree, nothing will happen.

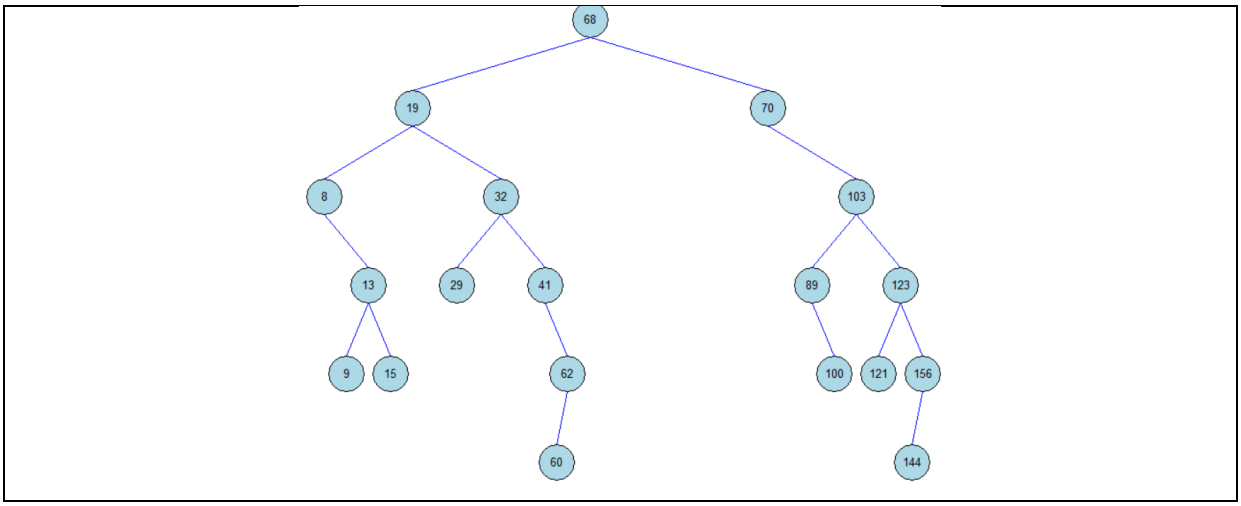


Table 2.2.7 Before Delete number 18

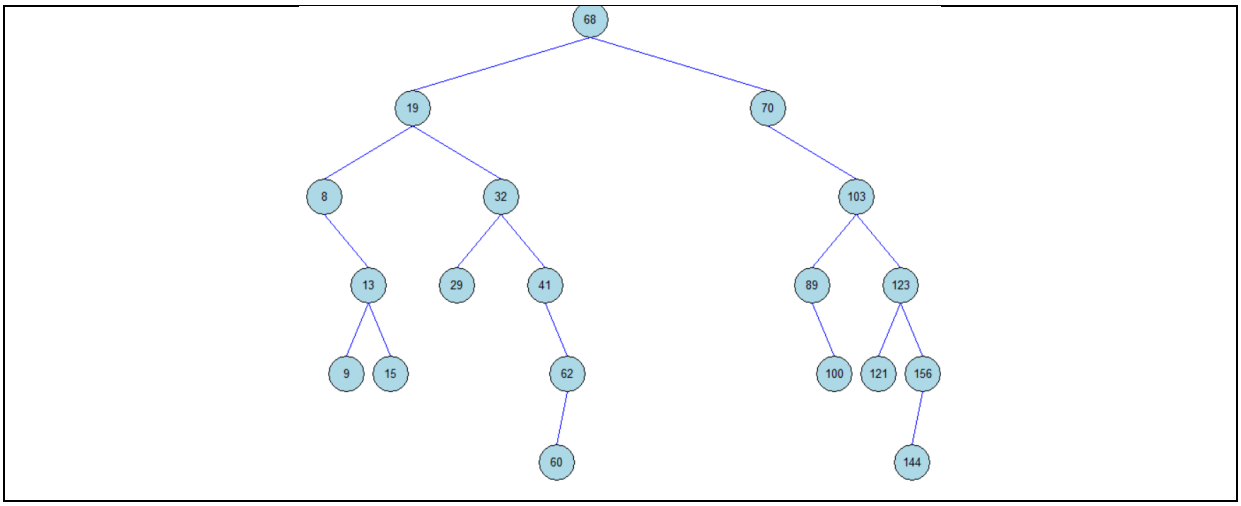


Table 2.2.8 After Delete number 18

Table 2.2.9 Message label "Key 18 is not found in the tree."

Find

Next, our third function is **Find**, it is used to locate a specific number within the tree. It will show whether a number is within the tree or not. To test this, we use the tree containing the 20 nodes created during the insertion phrase.

Test 1 – Find number 41 (within the tree).

After pressing the "Find" button, the system confirms that the number 41 is found in the tree, which means that the number is present and stored within the tree structure.

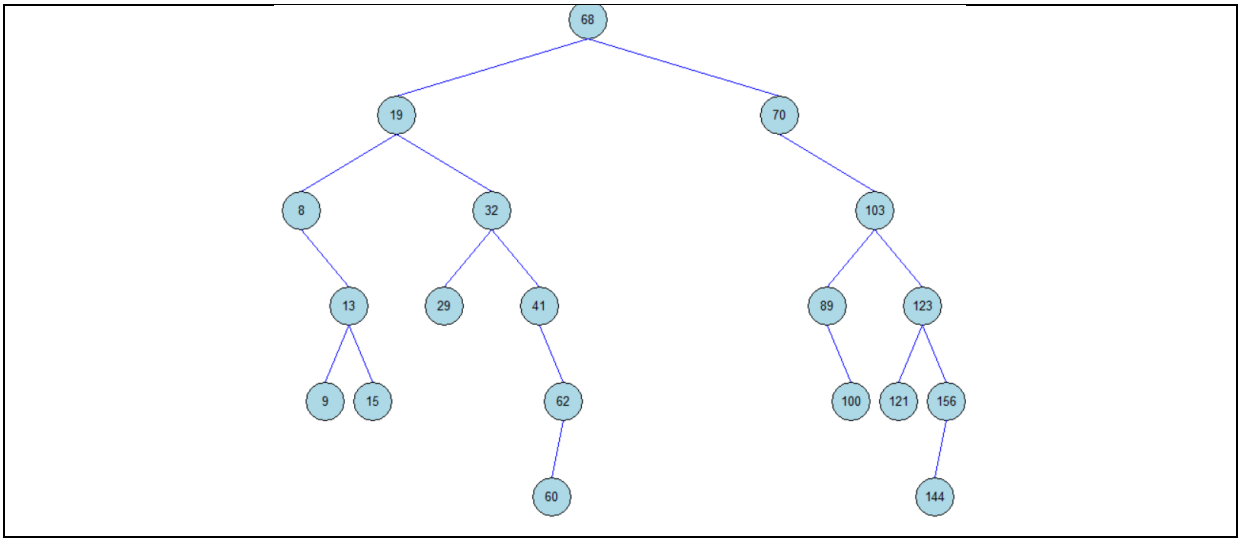


Table 2.3.1 Tree structure

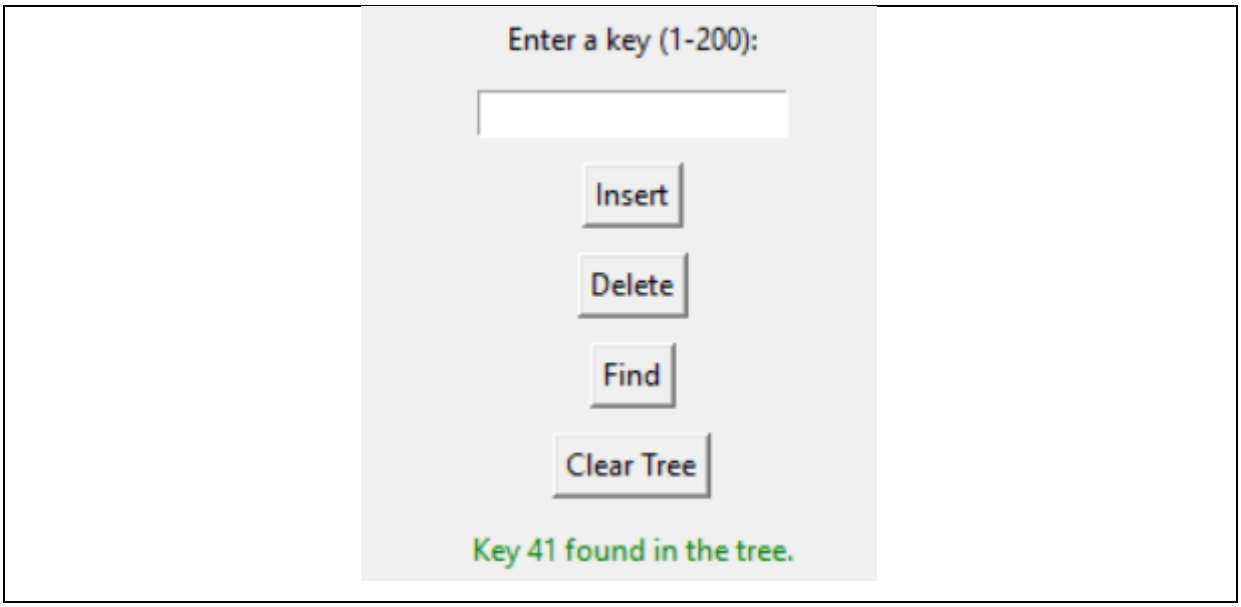


Table 2.3.2 Confirms the number

Test 2 – Find number 178 (not within the tree).

After pressing the “Find” button, the system confirms that the number 41 is not found in the tree, which means that the number is not present and thus not stored within the tree structure.

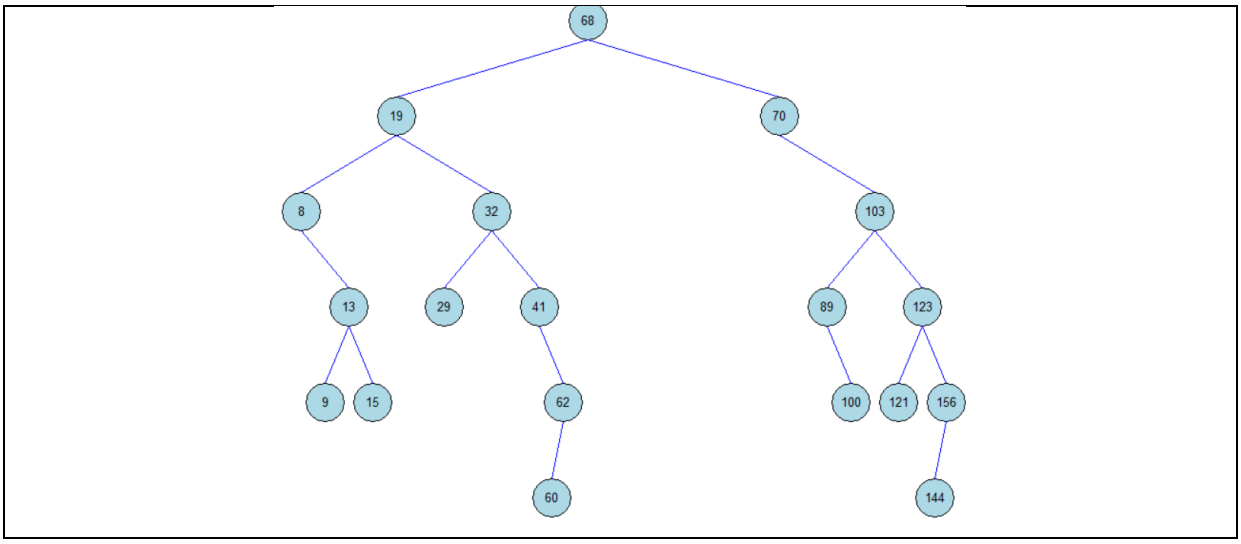


Table 2.3.3 Tree structure

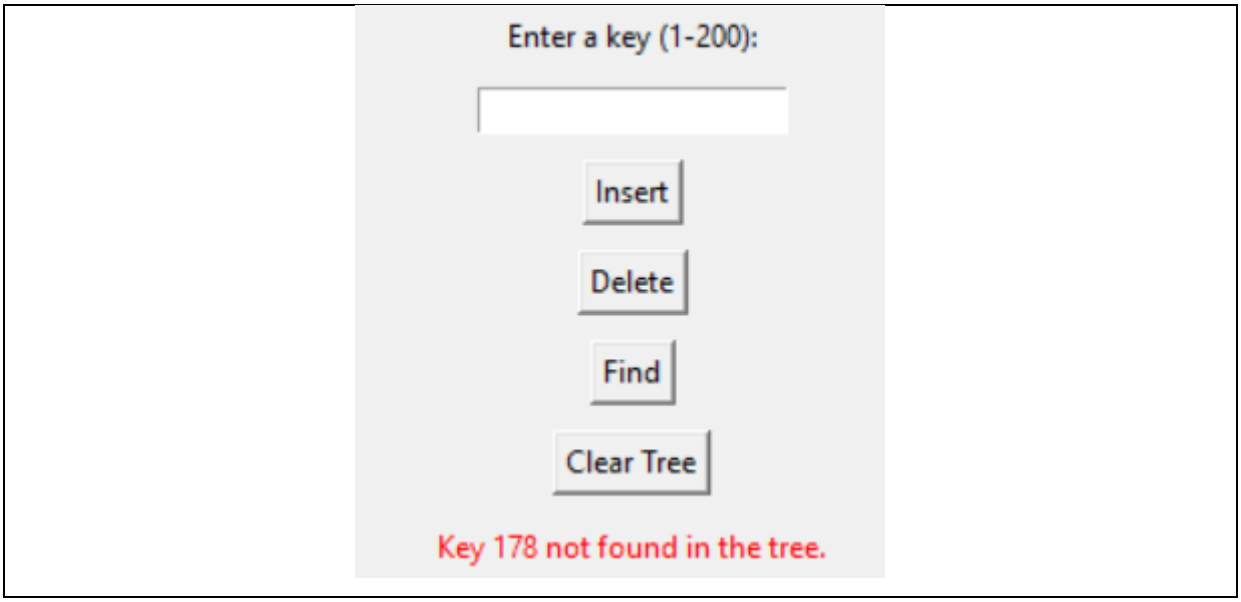


Table 2.3.4 Confirms the number

Clear Tree

Our last function in binary search tree is **Clear Tree**, it is used to remove all the nodes in the tree and resetting it to its initial state. To test this, we use the tree containing the 20 nodes created during the insertion phrase.

Test 1 – Clear every node in the tree

After pressing the “Clear Tree” button, the system removes all the nodes in the tree, resetting it to an empty state and a message will confirms this by displaying "Tree cleared successfully.".

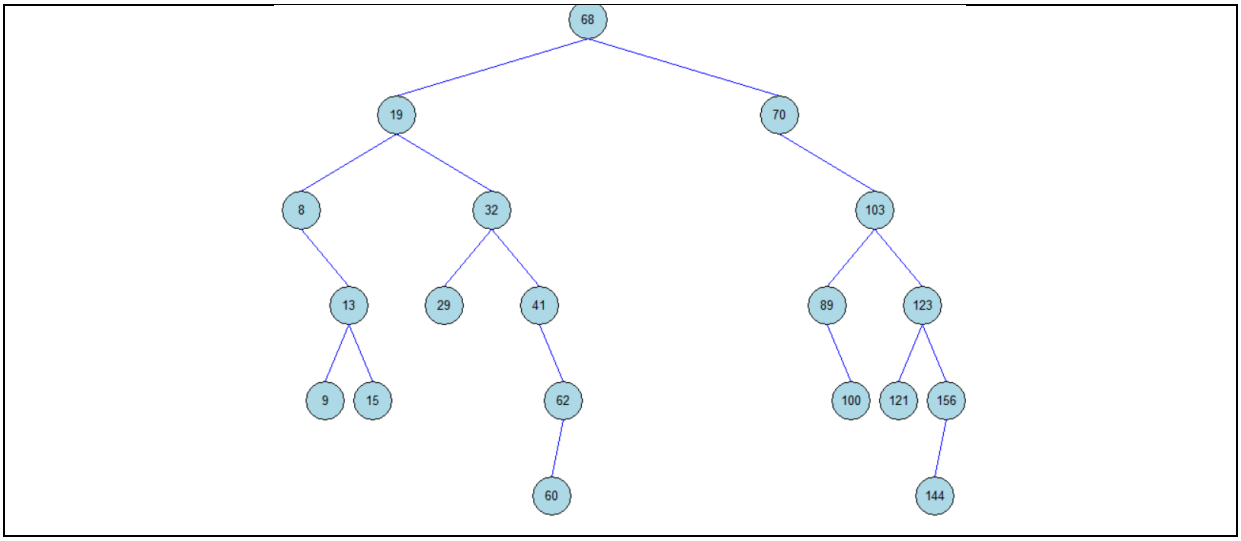


Table 2.4.1 Tree structure

Table 2.4.2 Clear tree

Code Breakdown:

In this part, I will briefly explain each part of the code.

1.
```python
import tkinter as tk
from tkinter import messagebox
```

Before writing our code, we need to import an additional library called “tkinter”, and from this library we specifically add in “messagebox”.

2.
```python
class TreeNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

The code begins with a class name TreeNode, which is used to represent a node. Each node has left and right attributes, the attributes are set to “None”, which indicates that in default there is no node to begin with.

3.
```python
class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = TreeNode(key)
        else:
            self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        if key < node.key:
            if node.left is None:
                node.left = TreeNode(key)
            else:
                self._insert_recursive(node.left, key)
        elif key > node.key:
            if node.right is None:
                node.right = TreeNode(key)
            else:
                self._insert_recursive(node.right, key)
```

This class is name as BinarySearchTree, we first define the tree root to be none, meaning that the tree is initially empty. Next, we put in insert method to add keys to the tree. If the tree is empty to begin with, the first insert key will be assigned as root node, else it will determine

whether the new insert key should put into the left or right subtree based on the value compare to the current nodes.

4.
```python
    def delete(self, key):
        self.root = self._delete_recursive(self.root, key)

    def _delete_recursive(self, node, key):
        if node is None:
            return node

        if key < node.key:
            node.left = self._delete_recursive(node.left, key)
        elif key > node.key:
            node.right = self._delete_recursive(node.right, key)
        else:
            if node.left is None:
                return node.right
            elif node.right is None:
                return node.left

            temp = self._find_min(node.right)
            node.key = temp.key
            node.right = self._delete_recursive(node.right, temp.key)

        return node
```

This is the delete function under the class BinarySearchTree, it searches key to be delete by passing the tree, moving left to right based on comparison with the current node's key. In case if the node to be delete has one child no matter it is left or right child, the child will replace the node. In case where the node has two children, finds the smallest key in the right subtree using _find_min, then the current node is replaced with this key and the original right node will be deleted to prevent duplication. Or if the key is not found, it returns with none.

5.
```python
    def find(self, key):
        return self._find_recursive(self.root, key)

    def _find_recursive(self, node, key):
        if node is None or node.key == key:
            return node
        if key < node.key:
            return self._find_recursive(node.left, key)
        return self._find_recursive(node.right, key)

    def _find_min(self, node):
        current = node
        while current and current.left is not None:
            current = current.left
        return current

    def clear(self):
        self.root = None
```

This is the find and clear function under the class BinarySearchTree, the find function search the key to be find by pass through the tree, moving left to right based on comparison with the current node's key. If the key is not found, it returns with none. While the clear function removes all node from the binary search tree, cleaning it and resetting it to an empty state.

6.

```python
class BinarySearchTreeGUI:
    def __init__(self, root):
        self.tree = BinarySearchTree()
        self.root = root
        self.root.title("Binary Search Tree Visualization")

        self.label = tk.Label(root, text="Enter a key (1-200):")
        self.label.pack(pady=5)

        self.entry = tk.Entry(root)
        self.entry.pack(pady=5)

        self.insert_button = tk.Button(root, text="Insert", command=self.insert_key)
        self.insert_button.pack(pady=5)

        self.delete_button = tk.Button(root, text="Delete", command=self.delete_key)
        self.delete_button.pack(pady=5)

        self.find_button = tk.Button(root, text="Find", command=self.find_key)
        self.find_button.pack(pady=5)

        self.clear_button = tk.Button(root, text="Clear Tree", command=self.clear_tree)
```

```python
        self.clear_button.pack(pady=5)

        self.message_label = tk.Label(root, text="", fg="red")
        self.message_label.pack(pady=5)

        self.canvas = tk.Canvas(root, width=800, height=800, bg="white", scrollregion=(0, 0, 1600, 1200))
        self.canvas.pack(fill=tk.BOTH, expand=True)

        self.x_scrollbar = tk.Scrollbar(root, orient=tk.HORIZONTAL, command=self.canvas.xview)
        self.x_scrollbar.pack(side=tk.BOTTOM, fill=tk.X)
        self.y_scrollbar = tk.Scrollbar(root, orient=tk.VERTICAL, command=self.canvas.yview)
        self.y_scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        self.canvas.config(xscrollcommand=self.x_scrollbar.set, yscrollcommand=self.y_scrollbar.set)

        self.root.bind('<Return>', self.handle_enter_key)
```

This class is name BinarySearchTreeGUI, this uses “tkinter” that we import at the begin to visualize the binary search tree. We name the title “Binary Search Tree Visualization”, put in GUI components such as input box to entering keys, buttons for insert, delete, find and clear nodes. We also added a message label that provide feedback or error messages. For example, we put in a label state that we only allow keys that is within 1 to 200. Additionally, we added horizontal and vertical scrollbars to allow navigation within a large tree. The GUI also bind in the Enter key to trigger operations by pressing Enter instead of clicking button.

7.

```python
    def handle_enter_key(self, event):
        self.insert_key()
```

This is a function under the class BinarySearchTreeGUI that allows users to insert key using Enter key.

8.
```python
    def insert_key(self):
        key = self.entry.get()
        if key.isdigit():
            key = int(key)
            if 1 <= key <= 200:
                self.tree.insert(key)
                self.display_tree()
                self.message_label.config(text="")
            else:
                self.message_label.config(text="Please enter a number between 1 and 200.", fg="red")
        else:
            self.message_label.config(text="Please enter a valid number.", fg="red")
        self.entry.delete(0, tk.END)
```

This is the insert_key function under the class BinarySearchTreeGUI, where user input a value into the input box. After an input is entered, the function check whether the value is a valid number or is between the number of 1 and 200. After a successful insertion, the display_tree method is called out to update the tree visualization on the canvas. If the input isn't a number, an error message "Please enter a valid number." is displayed in red. And if the number is out of the range, the message "Please enter a number between 1 and 200." is shown in red. Afterward, the input box is cleared so the user can enter a new value.

9.
```python
    def delete_key(self):
        key = self.entry.get()
        if key.isdigit():
            key = int(key)
            if self.tree.find(key):
                self.tree.delete(key)
                self.display_tree()
                self.message_label.config(text=f"Key {key} is deleted.", fg= "green")
            else:
                self.message_label.config(text=f"Key {key} not found in the tree.", fg="red")
        else:
            self.message_label.config(text="Please enter a valid number.", fg="red")
        self.entry.delete(0, tk.END)
```

This is the delete_key function under the class BinarySearchTreeGUI, where user enter a value into the input box. After an input is entered, the function check whether the value is a valid number. If valid, the find function is call out to check if the key exists in the tree. If the key is found, it is deleted with a message “Key {key} is deleted.” shown in green. If the key is not found, a message “Key {key} is not found.” is displayed in red. And if the function check that the value entered is not valid, a message “Please enter a valid number.” is display in red. Afterward, the input box is cleared so the user can enter a new value.

10.
```python
    def find_key(self):
        key = self.entry.get()
        if key.isdigit():
            key = int(key)
            node = self.tree.find(key)
            if node:
                self.message_label.config(text=f"Key {key} found in the tree.", fg="green")
            else:
                self.message_label.config(text=f"Key {key} not found in the tree.", fg="red")
        else:
            self.message_label.config(text="Please enter a valid number.", fg="red")
        self.entry.delete(0, tk.END)
```

This is the find_key function under the class BinarySearchTreeGUI, where user enter a value into the input box. After an input is entered, the function check whether the value is a valid number. If valid, the function check if the key exists in the tree. If the key is found, a message "Key {key} found in the tree." is shown in green. If the key is not found, a message "Key {key} not found in the tree." is displayed in red. And if the function check that the value entered is not valid, a message "Please enter a valid number." is display in red. Afterward, the input box is cleared so the user can enter a new value.

11.
```python
    def clear_tree(self):
        self.tree.clear()
        self.canvas.delete("all")
        self.message_label.config(text="Tree cleared successfully!", fg="green")
```

This is the clear_tree function under the class BinarySearchTreeGUI, when user press this button, all nodes in the binary search tree will be deleted, and clear canvas leaving it blank. Then, a message "Tree cleared successfully!" is shown in green.

12.
```python
    def display_tree(self):
        self.canvas.delete("all")
        if not self.tree.root:
            return

        canvas_width = self.canvas.winfo_width()
        start_x = canvas_width // 2
        self.y_offset = 100

        positions = {}
        self._assign_positions(self.tree.root, start_x, 20, canvas_width // 4, positions)

        self._draw_tree(self.tree.root, positions)
```

This is the display_tree function under the class BinarySearchTreeGUI, first it clears all existing elements on the canvas using self.canvas.delete("all") to prevent any previous node to visualize. If the tree is empty, the method returns without display anything. For non-empty trees, it calculates the canvas width to determine the root node's initial position and centre it horizontally. It also set up vertical to initialized to space nodes appropriately. Next, the

_assign_positions method is used to figure out the (x, y) positions for all nodes based on how far apart their sibling should be. These positions are saved in a dictionary. Finally, the _draw_tree method is called to draw the nodes and lines on the canvas, creating a clear picture of the tree.

13.
```python
    def _assign_positions(self, node, x, y, x_offset, positions):
        if node is not None:
            positions[node] = (x, y)
            self._assign_positions(node.left, x - x_offset, y + self.y_offset, x_offset // 2, positions)
            self._assign_positions(node.right, x + x_offset, y + self.y_offset, x_offset // 2, positions)
```

This is the assign_position function under the class BinarySearchTreeGUI, this uses to infinite loop to assign positions to each node in the tree. If the node already has a left or right child, it calculates their positions by shifting horizontally, either to the left or right, based on the current position of the parent node. The vertical position is adjusted by moving down a set amount, ensuring proper spacing between the levels of tree. These positions are then stored in a dictionary to be used later for display the tree on the canvas.

14.
```python
    def _draw_tree(self, node, positions):
        if node is not None:
            x, y = positions[node]
            r = 20

            self.canvas.create_oval(x - r, y - r, x + r, y + r, fill="lightblue")
            self.canvas.create_text(x, y, text=str(node.key), font=("Arial", 10))

            if node.left:
                x_left, y_left = positions[node.left]
                self.canvas.create_line(x, y + r, x_left, y_left - r, fill="blue")
                self._draw_tree(node.left, positions)
            if node.right:
                x_right, y_right = positions[node.right]
                self.canvas.create_line(x, y + r, x_right, y_right - r, fill="blue")
                self._draw_tree(node.right, positions)
```

This is the _draw_tree function under the class BinarySearchTreeGUI, this uses for drawing the binary search tree on the canvas. It first checks whether there is a node or not, if there is data, it draw a circle using self.canvas.create_oval() centred at (x, y) with the radius of 20 and fill it with the colour light blue. The node's key is display with "Arial" font and text size of 10. If the node has left or right, it calculates the positions for these children and draws lines connecting the current node to the left and/or right child by using the self.canvas.create_line() method. The function then repeatedly calls itself to draw the left and right subtrees, ensuring the entire tree structure is displayed. If there is no data, it will return without drawing anything.

15.

```python
if __name__ == "__main__":
    root = tk.Tk()
    bst_app = BinarySearchTreeGUI(root)
    root.mainloop()
```

The if __name__ = “__main__” is used to ensure that the program only executed when the script is run directly, not when it is imported as a module to another script. The root = tk.TK() is meant to provide a graphical user interface to the main window using tkinker library. The bst_app = BinarySearchTreeGUI(root) is used to set up the GUI to interact with binary search tree when user insert, delete, find keys or clear tree. Finally, the root.mainloop() starts the Tkinter event loop, which keeps the window open and responsive to user interactions, such as button clicks or key presses.

Part 3: Problem 2 Implementation: Self-Balancing Tree (AVL Tree)

AVL Tree is a self-balancing Binary Search Tree. In an AVL tree, the difference between the heights of the right and left subtrees of any node cannot be more than one. The time complexity of an AVL tree is O (log n). In this implementation, we randomly selected 20 numbers, integers ranging between 1 and 200, as our dataset. Table 3.1 below shows the dataset used to implement our AVL tree.

50	30	12	20	45
1	77	124	150	98
23	31	102	130	5
170	7	4	14	190

Table 3.1.1 20 Random Numbers

In this program, we have 6 functions: insert, delete, find, clear tree, show balance factors and close balance factors. Users only can enter integer values ranging between 1 to 200. Otherwise, an error message will be displayed.

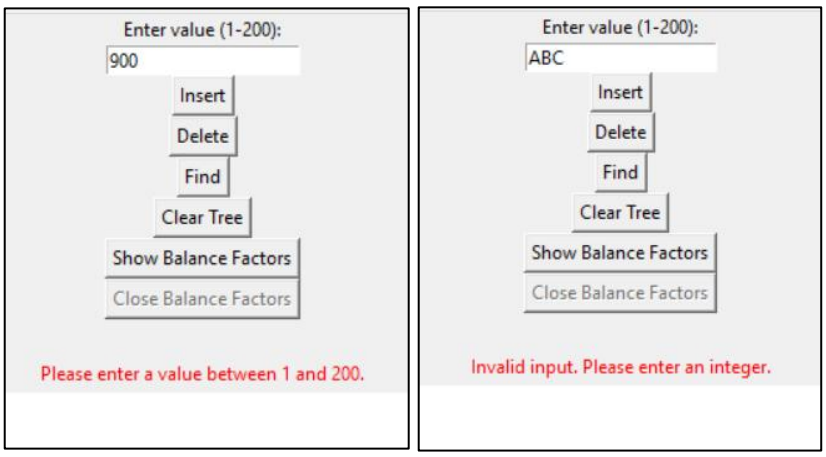


Diagram 3.1.2 Error Message Display

AVL trees exhibit the properties of binary search trees, but it has an additional property of dynamic self-balancing.

This is the additional rule listed for an AVL tree:

- **The difference between heights of left and right subtrees cannot be more than one for all nodes.** The difference is called the balance factor. If the balance factor is more than 1 or smaller than –1, rotation operation is needed.

There are four types of out-of-balance cases:

- Left-left: When the unbalanced node and its left child node are both left-heavy, a single right rotation should be performed on the unbalanced node.
- Right-Right: When the unbalanced node and its right child node are both right-heavy, a single left rotation should be performed.
- Left-Right: When the unbalanced node is left-heavy, and its left child node is right-heavy, two rotations need to be performed. First, perform a left rotation on the left child node, then perform a right rotation on the unbalanced node.
- Right-Left: When the unbalanced node is right-heavy, and its left child node is left-heavy, two rotations need to be performed. First, perform a right rotation on the left child node, then perform a left rotation on the unbalanced node.

Insertion

Step 1: Insert the 1st number: **50**

When the 1st number, 50 is entered. It is placed at the top of the tree, as the root node.

Diagram 3.1.3 Insert 1st number

Step 2: Insert the 2nd number: **30**

When the 2nd number, 30 is entered, it compares with the number 50 and is placed at the left subtree. This is because 30 is smaller than that of 50.

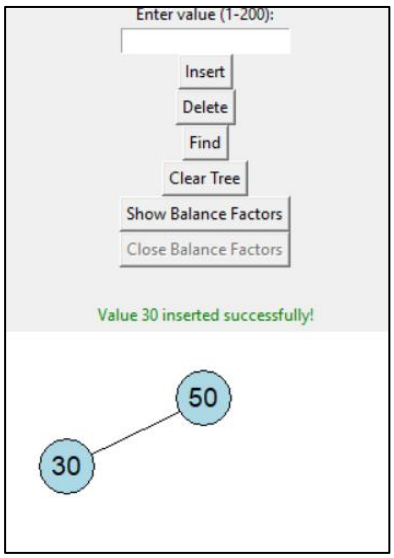


Diagram 3.1.4 Insert the 2nd number

Step 3: Insert the 3rd number: **12**

When the third number, 12 is inserted into the tree, it will first compare with 50 and 30. Since 12 is smaller than 50 and 30, it will be inserted as the left subtree of 30. However, the tree is unbalanced because the nodes 50 and 30 are left-heavy. In this case, a single right rotation is performed on node 50 to balance the tree.

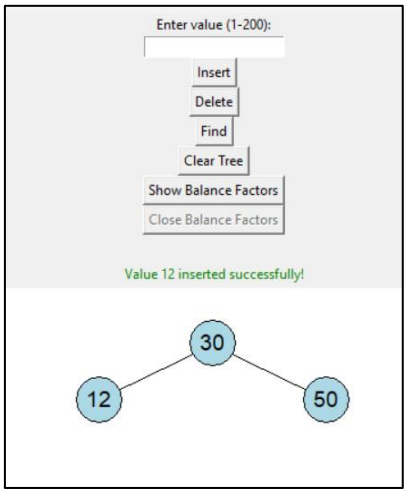


Diagram 3.1.5 Insert the 3rd number

Step 4: Insert the 4th to 8th number: **20, 45, 1, 77, 124**

When 20, 45, 1, 77, 124 are inserted into the tree, they will follow the rules of the AVL tree and insert to their respective positions.

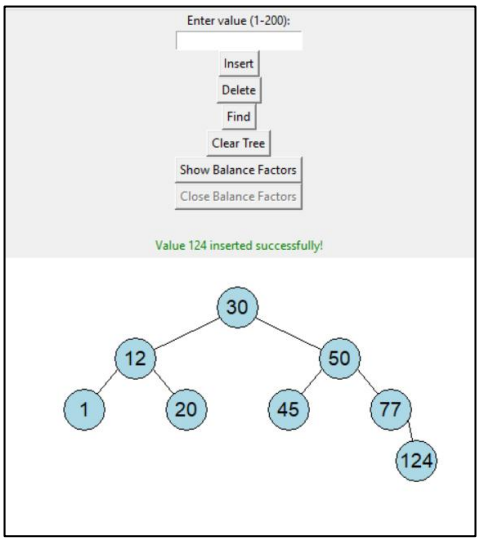


Table 3.1.6 Insert the 4th to 8th number

Step 5: Insert the 9th th number: **150**

When the 9th number, 150 is inserted into the tree, it is first placed at the right subtree of 124, this is because it is larger than that of 30, 50, 77 and 124. However, the node 77 will be unbalanced. Both the nodes 77 and 124 will be right-heavy. Thus, a single left rotation should be performed on 77 to get a balanced tree.

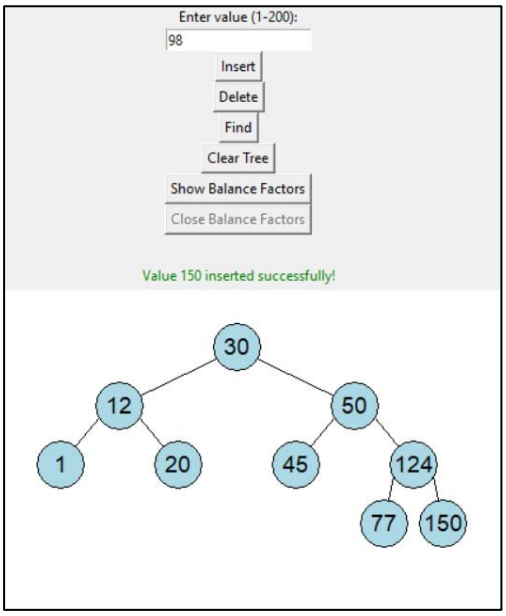


Diagram 3.1.7 Insert the 9th number

Step 6: Insert the 10th number: **98**

When the 9th number, 98 is inserted into the tree, it is placed at the right subtree of 77. This is because it is smaller than 124 but larger than 77. However, the tree is unbalanced because node 50 is right-heavy and node 124 is left-heavy. Right rotation is performed on node 124, followed by left rotation on node 50 to balance the tree.

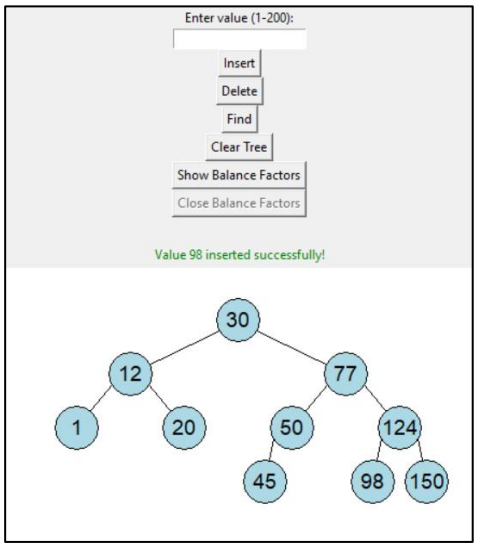


Diagram 3.1.8 Insert the 10th number

Step 7: Insert the 11th to 20th number: **23, 31, 102, 130, 5, 170, 7, 4, 14, 190**

As the numbers from 11th to 20th are inserted, the AVL tree will adjust its structure and perform rotations as needed to maintain balance.

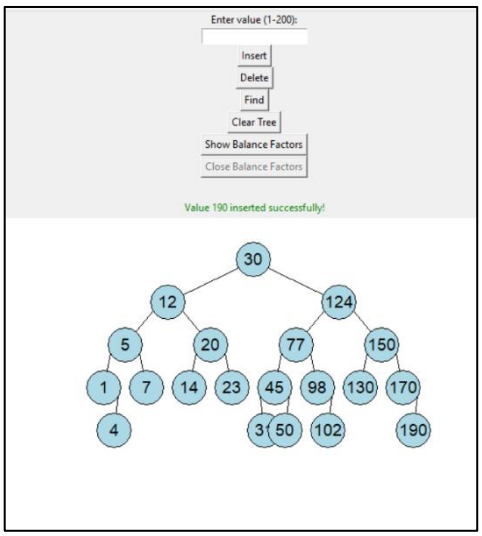


Diagram 3.1.9 Insert the 11th to 20th number

Deletion

1) Delete the number: **124**

   I have chosen number 124 to perform the deletion process. 124 is replaced by node 130 as it is the successor of 124. After 124 is deleted, both the nodes 150 and 170 are right-heavy. A single right rotation is performed on 150 to balance the tree.

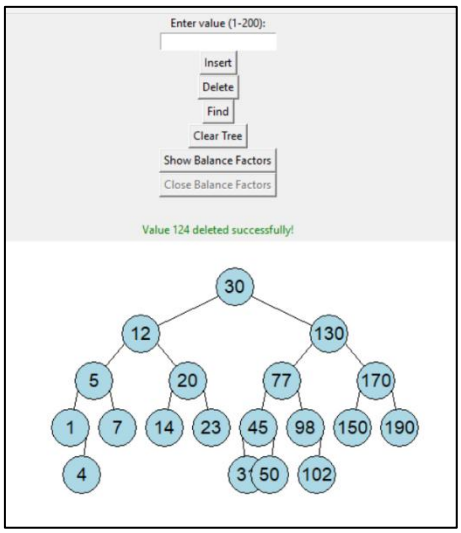


Diagram 3.2.0 Delete number 124

2) Delete the number: **77**

   Next, I have chosen number 77 to perform another deletion process. The node 77 is replaced by its successor, 98. After 77 is deleted, 102 will move up and become the right subtree of 98.

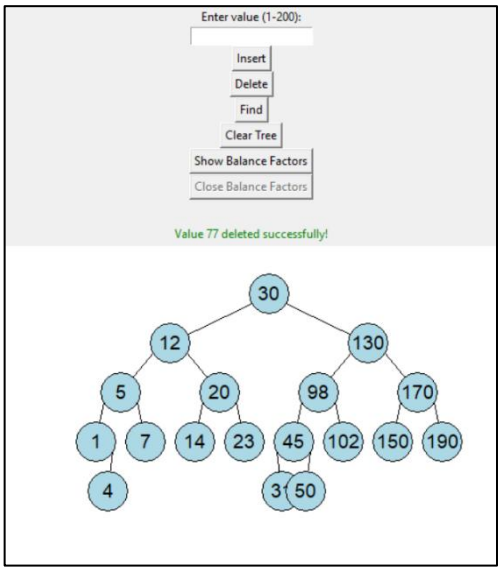
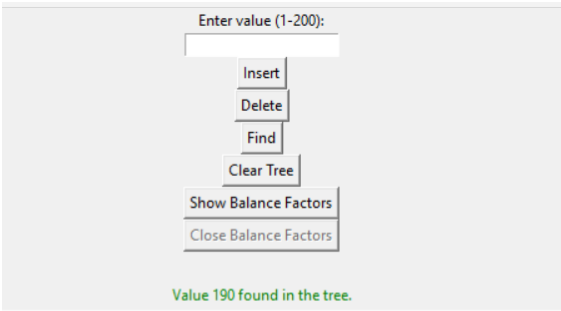


Diagram 3.2.1 Delete number 77

Find

I have chosen the number 190 to perform the find function. When the value 190 is inserted, it starts to compare with node 30, followed by 124, 150 and 170. When it reaches the node 190, the message "value 190 found in the tree".



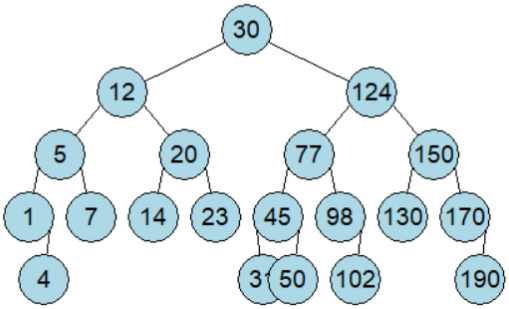
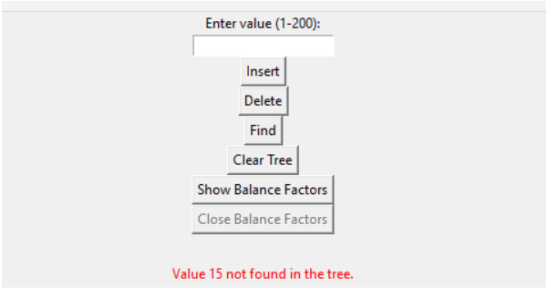


Diagram 3.2.2 Find number 190

If the value entered by user is not found in the tree, a red error message will be shown.



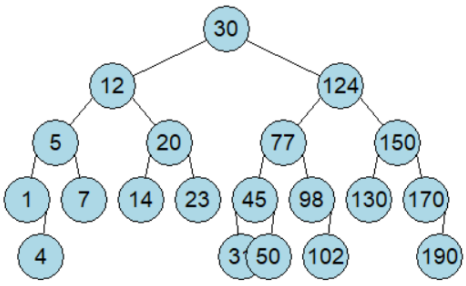


Diagram 3.2.3 Error Message

Code Explanation

```python
class Node:
    def __init__(self, key):
        self.key = key
        self.height = 1  # Node height
        self.left = None
        self.right = None
```

The code starts with a class representing a single node in an AVL tree, and each node has a key for its value, a height to keep track of the node's height. And left and right pointers for the child nodes.

```python
class AVLTree:
    def get_height(self, node):
        return node.height if node else 0

    def get_balance(self, node):
        if not node:
            return 0
        # Balance = Right height - Left height
        return self.get_height(node.right) - self.get_height(node.left)
```

This is the class for the AVL tree. The get_height function returns the height of the node, and returns 0 when the node is None. The get_balance function calculates the balance factor of the node by calculating right height - left height.

```python
    def right_rotate(self, y):
        x = y.left
        T2 = x.right
        x.right = y
        y.left = T2
        y.height = 1 + max(self.get_height(y.left), self.get_height(y.right))
        x.height = 1 + max(self.get_height(x.left), self.get_height(x.right))
        return x

    def left_rotate(self, x):
        y = x.right
        T2 = y.left
        y.left = x
        x.right = T2
        x.height = 1 + max(self.get_height(x.left), self.get_height(x.right))
        y.height = 1 + max(self.get_height(y.left), self.get_height(y.right))
```

This section of the code is about the left and right rotation of the tree. The right_rotate function is used to balance a left heavy tree. The left_rotate function is used to balance a right heavy tree.

4.

```python
    def insert(self, root, key):
        if not root:
            return Node(key)
        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.get_height(root.left), self.get_height(root.right))
        balance = self.get_balance(root)

        # Balance the tree
        if balance < -1 and key < root.left.key:  # Left-Left case
            return self.right_rotate(root)
        if balance > 1 and key > root.right.key:  # Right-Right case
            return self.left_rotate(root)
        if balance < -1 and key > root.left.key:  # Left-Right case
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if balance > 1 and key < root.right.key:  # Right-Left case
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
```

This function is about node insertion. It tells the program where to place the inserted nodes by determining if the key is equal, larger, or smaller than the root. It then updates the height of the ancestor nodes and balances the tree using rotations when necessary.

```python
    def delete(self, root, key):
        if not root:
            return root
        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if not root.left:
                return root.right
            elif not root.right:
                return root.left
            temp = self.get_min_value_node(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if not root:
            return root

        root.height = 1 + max(self.get_height(root.left), self.get_height(root.right))
        balance = self.get_balance(root)

        # Balance the tree
        if balance < -1 and self.get_balance(root.left) <= 0:
            return self.right_rotate(root)
        if balance < -1 and self.get_balance(root.left) > 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if balance > 1 and self.get_balance(root.right) >= 0:
            return self.left_rotate(root)
        if balance > 1 and self.get_balance(root.right) < 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root
```

This function is about deleting nodes. It first finds the node by comparing its size with the root and locate its position. After deleting, it replaces the space with the in-order successor and balances the tree using rotation.

```python
    def find(self, root, key):
        if not root:
            return None
        if key == root.key:
            return root
        elif key < root.key:
            return self.find(root.left, key)
        else:
            return self.find(root.right, key)
```

This function is about finding nodes. It compares the inserted node with the node and locates its position.

```python
# GUI Application
class AVLTreeGUI:
    def __init__(self, root):
        self.tree = AVLTree()
        self.root_node = None
        self.balance_factors_shown = False #State variable

        self.window = root
        self.window.title("AVL Tree Visualization")

        # User input for value
        self.label = tk.Label(root, text="Enter value (1-200):")
        self.label.pack()

        self.entry = tk.Entry(root)
        self.entry.pack()
```

This section focuses on the GUI of the code. The AVLTreeGUI creates a user interface using Tkinter. It initialises the window, labels and entry for visualisation.

```python
    # Buttons for operations
        self.insert_button = tk.Button(root, text="Insert", command=self.insert_value)
        self.insert_button.pack()

        self.delete_button = tk.Button(root, text="Delete", command=self.delete_value)
        self.delete_button.pack()

        # Button to find the node
        self.find_button = tk.Button(root, text="Find", command=self.find_value)
        self.find_button.pack()

        # Add "Clear Tree" button
        self.clear_button = tk.Button(root, text="Clear Tree", command=self.clear_tree)
        self.clear_button.pack()

        # Button to display balance factor
        self.balance_button = tk.Button(root, text="Show Balance Factors", command=self.toggle_balance_factors)
        self.balance_button.pack()

        # Close Balance Factors button
        self.close_balance_button = tk.Button(root, text="Close Balance Factors", command=self.close_balance_factors)
        self.close_balance_button.pack()
        self.close_balance_button.config(state=tk.DISABLED)  # Initially disabled

        # Label for displaying the find result
        self.find_result_label = tk.Label(root, text="", fg="blue", font=("Arial",12))
        self.find_result_label.pack()

        # Status message label
        self.message_label = tk.Label(root, text="", fg="red")
        self.message_label.pack()

        # Canvas for tree visualization
        self.canvas = tk.Canvas(root, width=800, height=600, bg="white")
        self.canvas.pack()
```

This section is about the initialisation of the buttons in the user interface, as well as the canvas.

```python
    def insert_value(self):
        try:
            value = int(self.entry.get())
            if 1 <= value <= 200:
                self.root_node = self.tree.insert(self.root_node, value)
                self.entry.delete(0, tk.END)
                self.draw_tree()
                self.message_label.config(text=f"Value {value} inserted successfully!", fg="green")
            else:
                self.message_label.config(text="Please enter a value between 1 and 200.", fg="red")
        except ValueError:
            self.message_label.config(text="Invalid input. Please enter an integer.", fg="red")
        finally:
            self.find_result_label.config(text="")

    def delete_value(self):
        try:
            value = int(self.entry.get())
            if self.tree.find(self.root_node, value) is None:
                self.message_label.config(text=f"Value {value} not found in the tree.", fg="red")
            else:
                self.root_node = self.tree.delete(self.root_node, value)
                self.message_label.config(text=f"Value {value} deleted successfully!", fg="green")
                self.entry.delete(0, tk.END)
                self.draw_tree()
            self.find_result_label.config(text="") #clear previous results
        except ValueError:
            self.message_label.config(text="Please enter a valid integer.", fg="red")
```

The two functions handle the insertion and deletion operation via the GUI. The insertion function retrieves the input value, validates it, and updates the tree. It gives messages when the insertion is successful in green. If the value is not in the specified range or is invalid, it issues a red message. The delete function also retrieves the inserted node, finds the node and deletes it. If the node is not found or is invalid, it will issue a message. A green coloured message will also be issued when the deletion is successful.

```python
    def find_value(self):
        try:
            value = int(self.entry.get())
            result = self.tree.find(self.root_node, value)
            self.entry.delete(0, tk.END)
            if result:
                self.message_label.config(text= f"Value {value} found in the tree.", fg="green")
            else:
                self.message_label.config(text=f"Value {value} not found in the tree.", fg="red")
        except ValueError:
            self.message_label.config(text="Please enter a valid integer.", fg="red")
```

This function finds the node in the tree, if the node is found, a green message will appear, otherwise, a red message will appear. An invalid red message will appear when the inserted value is not valid.

```python
    def clear_tree(self):
        self.root_node = None
        self.canvas.delete("all")
        self.message_label.config(text="Tree cleared successfully!", fg="green")
```

The clear tree function allows the user to clear the canvas. A success message will be showed in green afterwards.

```python
    def clear_tree(self):
        self.root_node = None
        self.canvas.delete("all")
        self.message_label.config(text="Tree cleared successfully!", fg="green")
        self.find_result_label.config(text="") #clear any previous find results
```

This draw_tree function clears the canvas and redraws the tree.

```python
    def _draw_tree_recursive(self, node, x, y, x_offset, y_offset):
        if node:
            # Increase the node radius for better spacing
            node_radius = 20

            # Draw the node as a circle
            self.canvas.create_oval(x - node_radius, y - node_radius, x + node_radius, y + node_radius, fill="lightblue")

            # Draw the node key in the center of the circle
            self.canvas.create_text(x, y, text=str(node.key), font=("Arial", 14))

            # Draw the left child
            if node.left:
                # Calculate the coordinates for the bottom-left edge of the node circle
                start_x = x - node_radius * math.cos(math.radians(30))
                start_y = y + node_radius * math.sin(math.radians(30))
                x_left = x - x_offset
                y_child = y + y_offset
                self.canvas.create_line(start_x, start_y, x_left, y_child)
                self._draw_tree_recursive(node.left, x_left, y_child, x_offset // 2, y_offset)

            # Draw the right child
            if node.right:
                start_x = x - node_radius * math.cos(math.radians(150))
                start_y = y + node_radius * math.sin(math.radians(150))
                x_right = x + x_offset
                y_child = y + y_offset
                self.canvas.create_line(start_x, start_y, x_right, y_child)
                self._draw_tree_recursive(node.right, x_right, y_child, x_offset // 2, y_offset)
```

The draw_tree_recursive function uses recursion to draw each node, and its connections based on the tree structure. The node keys are drawn in the centre of each circle and the lines are connected from a certain point to ensure neat alignment.

```python
    def toggle_balance_factors(self):
        if not self.balance_factors_shown:
            self.balance_factors_shown= True
            self.show_balance_factors()
            self.balance_button.config(state=tk.DISABLED) #Disable "show" button
            self.close_balance_button.config(state=tk.NORMAL) #Enable "Close" button

    def close_balance_factors(self):
        self.balance_factors_shown= False
        self.draw_tree() #Redraw the tree without balance factors
        self.balance_button.config(state=tk.NORMAL)
        self.close_balance_button.config(state=tk.DISABLED)

    def show_balance_factors(self):
        self.canvas.delete("all")
        if self.root_node:
            self._draw_tree_with_balance(self.root_node, 400, 50, 100)
```

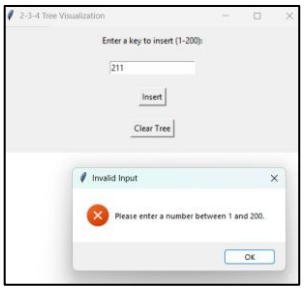
These functions are about the balance factors of the nodes. The close_balance_factors redraws the tree without balance factors and the show_balance_factor draws the tree with balance factors.

```python
    def _draw_tree_with_balance(self, node, x, y, x_offset):
        if node:
            # Increase the node radius for better spacing
            node_radius = 20

            # Calculate balance factor
            balance = self.tree.get_balance(node)

            # Draw the node as a circle
            self.canvas.create_oval(x - node_radius, y - node_radius, x + node_radius, y + node_radius, fill="lightblue")

            # Draw the node key and balance factor in the center of the circle
            self.canvas.create_text(x, y, text=f"{node.key}\n({balance})", font=("Arial", 12))

            # Draw the left child
            if node.left:
                x_left = x - x_offset
                y_child = y + 100
                self.canvas.create_line(x, y, x_left, y_child)
                self._draw_tree_with_balance(node.left, x_left, y_child, x_offset // 2)

            # Draw the right child
            if node.right:
                x_right = x + x_offset
                y_child = y + 100
                self.canvas.create_line(x, y, x_right, y_child)
                self._draw_tree_with_balance(node.right, x_right, y_child, x_offset // 2)
```

This function tells the program precisely how to draw the balance factors in the tree. It specifies the calculation and position of the balance factors.

Part 4: Problem 3 Implementation: B-Tree (2-3-4 Tree)

Balanced Tree (B-Tree) is a perfectly balanced tree and a type of the M-Tree. In this implementation, we will use 2-3-4 tree. From our code, we try to avoid people misused it. We set the integer limited from 1 to 200 and does not allow people repeat the same number.



We will need to have a dataset or array to perform B-Tree. The following dataset displays the number that we will use in the graph that is generated from our code.

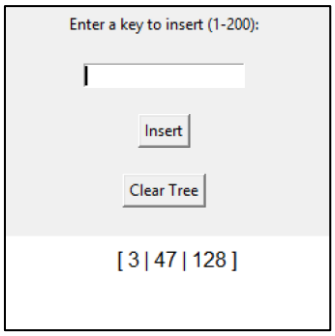
128	47	3	175	92
64	189	54	11	158
33	105	197	82	26
140	69	10	109	123

Table 4.1 - *20 Random Number*

There are few rules we need to know about for 2-3-4 tree:

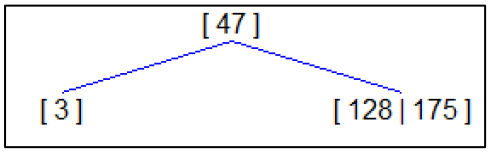
1) A 2-node has one data element and maximum two child nodes (GeeksforGeeks, 2022).
2) A 3-node has two data elements and maximum three child nodes (GeeksforGeeks, 2022).
3) A 4-node has three data elements and maximum four child nodes (GeeksforGeeks, 2022).
4) Each node is sorted from smallest to greatest, where 13 will be at the left of 15.

Step 1 – Insert first three number: **128, 47 and 3**



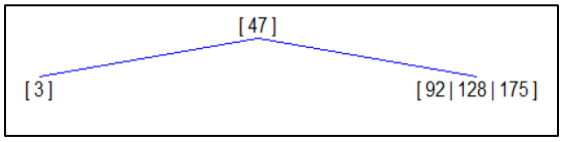
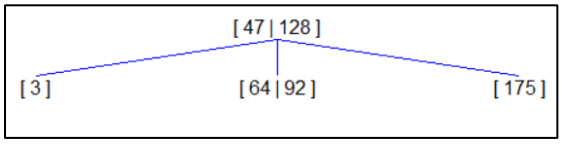
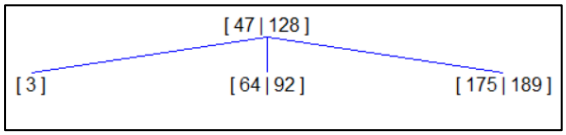
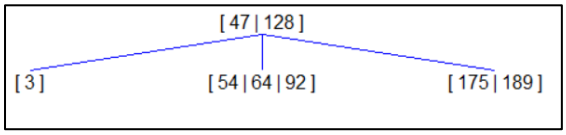
Step 2 – Insert 4th number: 175

As we insert the 4th number, the root node becomes [3 | 47 | 128 | 175]. In 2-3-4 Tree, this becomes overfilled. Thus, we find our middle number ,47 and make it become the new root. And the 3, smaller than 47 will go to the left. Meanwhile, 128 and 175 will go to the right.



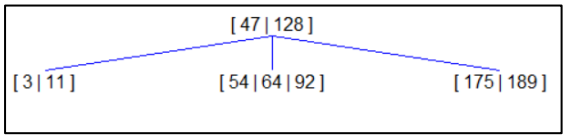
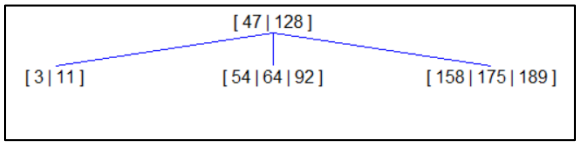
Step 3 – Insert 5th to 8th number: **92, 64, 189 and 54**

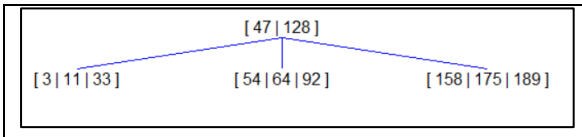
When we insert the 6th number, the [64 | 92 | 128 | 175], the code pushes the middle number, 128 to the root, and the number on the left will go to node between 47 and 128, and the number on the right go to right side. Meanwhile 7th and 8th numbers will continue insert to the node they should stay in.

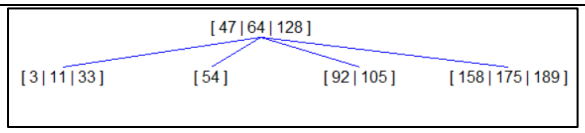
Insert 5th number: **92** 	Insert 6th number: **64** 
Insert 7th number: **189** 	Insert 8th number: **54** 

Step 4 – Insert 9th to 12th number:

We continue insert 9th number to 11th number. At 12th number, 105, becomes [54 | 64 | 92 | 105]. We will push the middle number, 64, to the root and separate them.

Insert 9th number: **11** 	Insert 10th number: **158** 
Insert 11th number: **33**	Insert 12th number: **105**

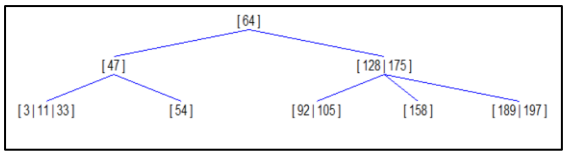




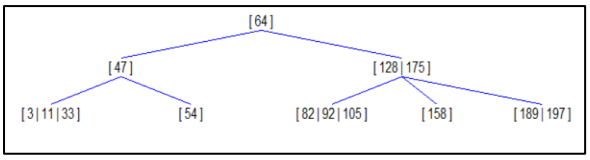
Step 5 – Insert 13th to 16th number:

We continue insert 13th number, 197, becomes [158 | 175 | 189 | 197] and 175 will push to upside, combine with 128. The following 3 numbers continue insert and do not change the tree shape.

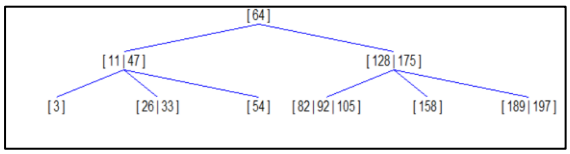
Insert 13th number: **197**



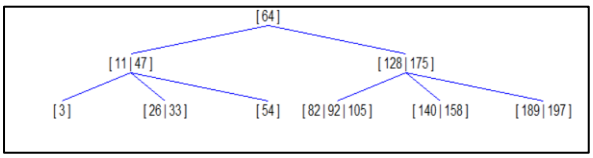
Insert 14th number: **82**



Insert 15th number: **26**



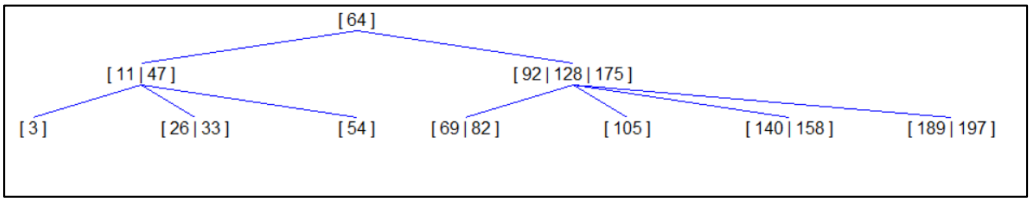
Insert 16th number: **140**



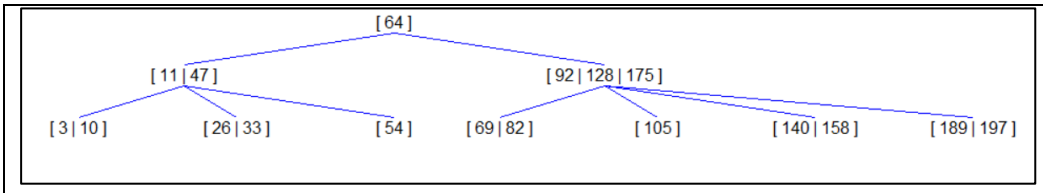
Step 6 – Insert 17th to 20th number:

We continue insert 69 to [82 | 92 | 105] become [69 | 82 | 92 | 105]. This causes the middle number, 92 to move above become [92 | 128 | 175]. After insert 10, the number 109 goes to [105] and becomes [105 | 109]. Lastly, we insert our final number, 123 and it goes down between 64 and 128, and go to the right leaf of the 92 becomes [105 |109| 123]. Finally, the final B-Tree is done.

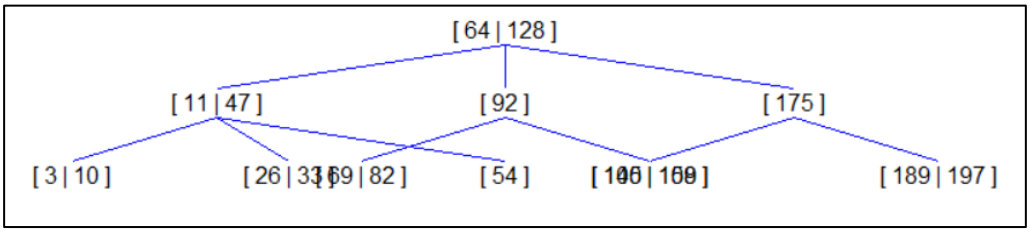
Insert 17th number: **69**



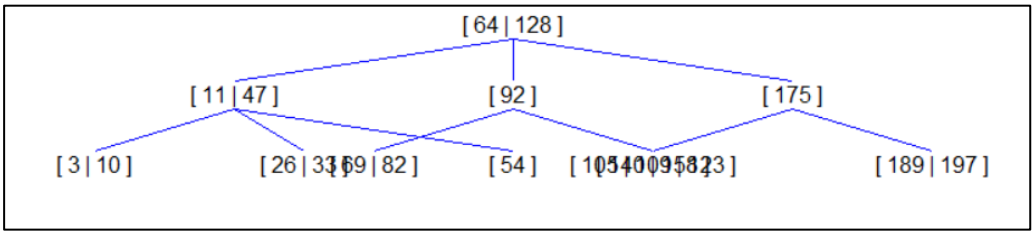
Insert 18th number: **10**



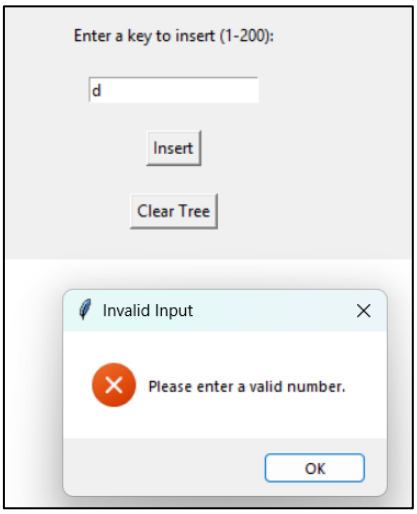
Insert 19th number: 109



Insert 20th number: 123



However, there are some limitations of our code to run a binary tree. First, the problem occurred when we inserted 19th and 20th numbers to the tree. The code causes the tree elements to clash together that makes it hard for people to read the number. Besides that, we are having issues to settle the alphabet style B-tree as it clashes with the number B-tree. Thus, we have disabled the alphabet input.



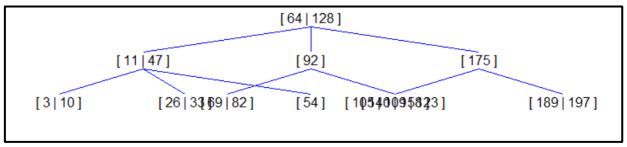
Part 5: Individual Member Contribution

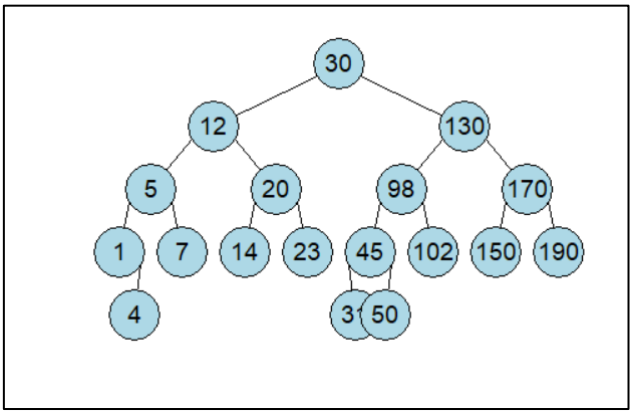
No.	Name	Student ID	Roles or distribution
1	Liang Yrenn Da	22067128	Binary Search Tree, Challenges
2	Ooi Jin Yon	22039069	B-Tree (2-3-4 Tree), Challenges
3	Isabelle Yau Li Shuang	23021165	Self -balancing tree (AVL Tree), proofreading
4	Peggy Liow En Ki	23020852	Self -balancing tree (AVL Tree), Challenges
5	Leow Yan Yi	23020324	Introduction, Conclusion

Part 6: Challenges

Challenges 1:

To begin with, we all have the same problem for all implementation, which is the illustration issues. When the tree has more depth, it causes a clash issue like the diagram below:





This issue happened due to the limitation in GUI code, as we were unable to properly adjust the spacing between each node to their position appropriately. To address this, we would need to improve our understanding of GUI program to refine our code.

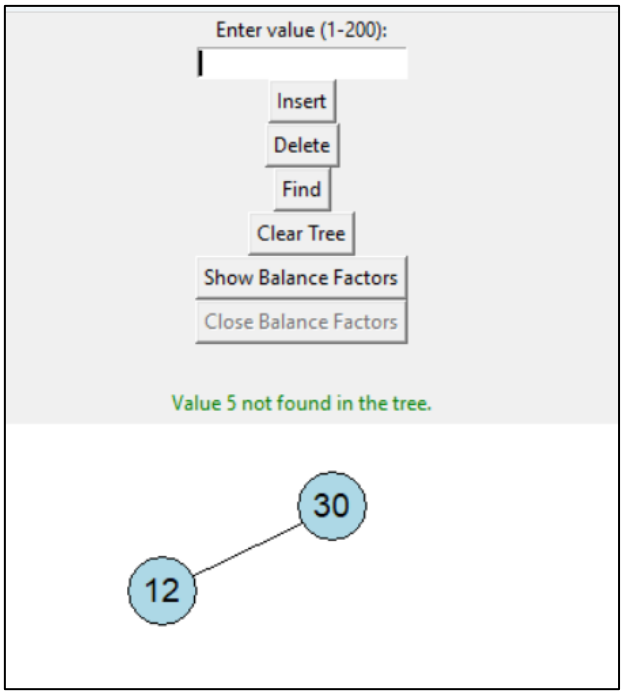
Challenges 2:

We are not able to capitalize the key_handler_function fully in our Binary Search Tree code. Although the Enter key is successfully bound to the GUI, it currently only works for insertion operation and not for other function such as deletion, find and clear tree.

Challenges 3:

In the AVL tree part, initially we have encountered a problem where the message will not disappear when the next operation is taken (in insertion, deletion, and find). For example, the message “Value 5 not found in the tree” still exists after we insert node 12. To solve this problem, we have modified the code in insert_value and delete_value by adding the

confirmation messages “Value {value} inserted successfully!” and “Value {value} deleted successfully”, so that the previous message will not stay there after the next operation is taken.



Part 7: Conclusion

This report aimed to explore the Binary Search Tree (BST), AVL Tree and B-Tree (2-3-4 Tree). We explore the way that each structure solves some challenges related to data organization and manipulation during the process. We also know that different data structures will have various features and characteristics. For example, the BST serves as a principle of understanding tree-based algorithms. The AVL tree states the importance of maintaining balance for having good performance while operating. B-tree is also used to explore the multi-way search to manage large datasets with efficiency.

While implementing the data structure, we faced various challenges which included GUI visualization issues and debugging algorithms for self-balancing. Nevertheless, these issues reflect our teamwork since they enhanced our team collaboration. Besides, they also magnify our problem-solving and technical skills. To ensure our project is completed successfully, we contribute and achieve our target by giving full play to our strengths.

In conclusion, this project helps us to enhance our understanding of data structures and algorithms by combining theory with practical work. It highlights the importance of selecting suitable data structures while operating. It also points out the importance of teamwork while facing challenges. All of the knowledge that we gained from this project may help us in the future while optimizing data structure and other related scenarios.

Part 8: References

GeeksforGeeks. (2022). *2-3-4 Tree*. GeeksforGeeks. https://www.geeksforgeeks.org/2-3-4-tree/